SARVAJANIK COLLEGE OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER ENGINEERING

B. Tech III, Semester V, Division II (Odd Semester 2026–27)

Subject: Web Technologies (BTCO13502)

OPEN-ENDED PROJECT (OEP) COMPREHENSIVE REPORT

MEDICYCLE

Smart Medical Supply Redistribution Management System

Decentralized Non-Drug Consumable Recovery, Clinic Requisition & Secure Handshake Platform

Group No.: 7 | Submission & Evaluation Period: 26/10/2026 to 30/10/2026

Live Deployed URL: **http://medicycle.infinityfreeapp.com**

GitHub Source Repository: https://github.com/preygoti/medicycle

Submitted by Group 7 Members:

Sr.	Student Name	Enrollment No.	Assigned Module Responsibilities
1	Rudra Bhuva	ET24BTCO145	System Architect & Database Lead (3NF Schema, PDO, Core Sessions)
2	Yagnen Ghodadara	ET24BTCO151	Frontend UI/UX Designer (Bootstrap 5.3 Theme, Modals, Responsive Layout)
3	Rases Gorasiya	ET24BTCO154	Healthcare Supplier Portal Lead (Batch Inventory CRUD, CSV Import, Approvals)
4	Utsav Gorasiya	ET24BTCO155	NGO & Clinic Portal Lead (Catalog Search & Multi-filter, Need Requisitions)

5	Prey Goti	ET24BTCO156	Authentication & Security Lead (2FA Email OTP, QR Pass & 6-Digit PIN Handshake)

1. Introduction

1.1 Background & The Medical Waste Paradox

Modern healthcare facilities worldwide generate massive volumes of unexpired, sterile, and non-drug medical consumables that are prematurely incinerated or landfilled. Hospitals, diagnostic centers, and wholesale distributors purchase consumables—such as surgical gloves, N95 respirators, sterile gauze swabs, syringes, and IV administration sets—in massive bulk quantities to hedge against procurement shortages. Due to shifts in patient volume, changing clinical protocols, and strict hospital storage clearance cycles, extensive volumes of perfectly usable supplies are declared surplus months prior to their manufacturer-certified expiration dates.

Concurrently, grassroots charitable healthcare providers—including community outreach dispensaries, rural healthcare centers, charitable clinics, and disaster-relief NGOs—operate under severe operational deficits. These non-profit providers routinely lack adequate supplies of basic personal protective equipment (PPE) and sterile dressings. A stark institutional paradox emerges: one sector of the healthcare ecosystem expends significant resources incinerating usable sterile supplies, while another sector struggles to deliver basic clinical care due to acute supply deficits.

This critical imbalance is not caused by a deficit of donor goodwill, but rather by systemic information asymmetry, lack of inventory visibility, and absence of mutual verification protocols. Donors lack visibility into local clinic requirements, clinics lack visibility into supplier surplus inventories, and both parties require tamper-evident verification ensuring supplies are neither stolen nor redirected into unauthorized commercial channels. MediCycle was engineered to bridge this gap through a decentralized, transparent web redistribution platform.

1.2 Problem Statement & Objectives

Formal Definition: Problem Statement:

To design, architect, and deploy an open-ended, secure, database-driven web application enabling verified healthcare suppliers to declare surplus, unexpired, non-drug medical consumables, empowering charitable clinics and NGOs to discover and requisition supplies based on real-time urgency, and guaranteeing authenticated, physical handover verification through a secure 6-digit PIN and dynamic QR code handshake mechanism.

Core Technical Objectives:

- **1. Syllabus Compliance:** Deliver a complete working web application built strictly upon the course stack: Vanilla PHP 8.2, MySQL relational database (InnoDB), HTML5, CSS3 with Bootstrap 5.3, and ES6 JavaScript.
- **2. Database Integrity:** Implement full Database CRUD (Create, Read, Update, Delete) and multi-parameter instant search across normalized tables.
- **3. Resilient Validation:** Ensure two-tier input validation (HTML5/JavaScript client-side + strict PHP PDO server-side sanitization) across every input field.
- **4. Session Security:** Provide robust session management with HTTP-only cookie flags, inactivity timeouts, session regeneration, and role-based page guards.

- **5. Streamlined Roles:** Enforce a strict two-role operational model (Healthcare Supplier and NGO/Clinic), eliminating complex external courier intermediaries in favor of a direct mutual physical handover protocol.

1.3 Scope & Healthcare Safety Protocol

To eliminate liability, avoid regulatory cold-chain complexities, and prevent illicit drug diversion, MediCycle strictly enforces a Non-Drug Medical Consumable Boundary. The application strictly prohibits prescription pharmaceuticals, injectables containing active compounds, and controlled narcotics.

Category Domain	Eligible Items (Strictly In-Scope)	Prohibited Items (Strictly Out-of-Scope)
Personal Protective Equipment	Nitrile/latex examination gloves, surgical face masks, N95 respirators, disposable surgical gowns, face shields.	Used, torn, unsealed, or contaminated protective wear.
Wound Care & Sterile Dressings	Sterile gauze pads, elastic crepe bandages, surgical tape, dressing sponges, non-adherent pads.	Dressings with broken seals or moisture-damaged packaging.
Non-Drug Administration	Capped sterile syringes, IV cannulas, infusion sets, butterfly needles, administration tubing.	Any pre-filled syringe containing medication, blood products, vaccines.
Diagnostic Disposables	Urine collection cups, lancets, disposable probe covers, non-chemical diagnostic collection swabs.	Prescription diagnostic test kits requiring regulated cold-chain refrigeration.
Mobility & First Aid Consumables	Triangular bandages, arm slings, cervical collars, splints, emergency thermal foil blankets.	Controlled narcotics, scheduled pharmaceuticals, refrigerated biologicals.

1.4 Technology Stack Justification

- **Backend:** Vanilla PHP 8.2 (Procedural & MVC Architecture): Serves as the robust backend engine. Provides native session management, strict typing, match expressions, and PDO prepared statements eliminating SQL injection vulnerabilities. Natively supported across zero-cost university production environments (InfinityFree / Apache).
- **Database:** MySQL 8.0 / MariaDB (InnoDB Storage Engine): Provides true relational data management with primary keys, foreign key constraints with ON DELETE CASCADE / RESTRICT actions, composite indexes, and row-level locking (SELECT ... FOR UPDATE) inside ACID transactions.
- **Frontend Styling:** Bootstrap 5.3 & Custom Responsive CSS: Modern 12-column flexbox grid, clean healthcare teal color system (#0f766e), mobile navigation drawer, and frosted glass modal overlays (backdrop-filter: blur(12px)) without legacy jQuery dependencies.

- **Client Scripting:** Vanilla JavaScript (ES6+), Fetch API & QRCode.js: Executes debounced catalog filtering, real-time AJAX event polling, camera-based QR decoding (html5-qrcode), and single-page collection pass rendering.

2. System Modules & Architecture

MediCycle is partitioned into five tightly coupled, highly cohesive functional modules:

2.1 Module 1: User Authentication, 2FA Email OTP & Session Security

Led by Rudra Bhuva and Prey Goti, this module handles identity verification and session defense.

- **Registration Controller:** Role-Based Registration (register.php): Visitors register under exactly one of two legal entity roles: Healthcare Supplier or NGO / Charitable Clinic. The controller validates 11 input attributes (Contact Person, Email, Phone, Password, Organization Name, Entity Type, License Number, Street Address, City, State, Pincode). Passwords are encrypted using password_hash(..., PASSWORD_BCRYPT). Newly registered accounts are stored with email_verified = 0 and status = 'inactive'.
- **2FA Verification Engine:** Two-Factor Email OTP Activation (verify-email.php): Upon registration, the mail service (includes/mail.php) invokes generate_six_digit_otp() generating a cryptographically random 6-digit code via random_int(100000, 999999). The plain OTP is encrypted using bcrypt and stored in email_otps.otp_hash with a strict 10-minute validity window. Users enter the code across 6 auto-advancing input fields. A 45-second resend cooldown prevents email spamming, and a 5-attempt threshold locks the code against brute-force attacks.
- **Session Defense & Multi-Account:** Secure Session Manager (includes/session.php): Establishes HttpOnly, SameSite=Lax session cookies. Invokes session_regenerate_id(true) on login to defend against session fixation. Enforces an automatic 7200-second (2-hour) inactivity timeout. Implements multi-account context switching allowing developers and testers to maintain active Supplier and NGO logins concurrently across browser tabs.

2.2 Module 2: Healthcare Supplier Portal & Batch Inventory CRUD

Led by Rases Gorasiya, this module provides medical donors with inventory management capabilities.

- **Surplus Ingestion:** Surplus Listing & Declaration Form (supplier/add-supply.php): Allows authorized hospital staff to list surplus consumables. Captures Supply Name, Category ID, Quantity, Packaging Unit, Listing Type (Regular/Surplus/Disposal), Condition Status (New/Unopened vs Sterile Sealed), Packaging Integrity, Batch Number, Expiry Date, Storage Specs, and Facility Location. Rejects past or today's expiry dates on the server side.
- **Priority Scoring Algorithm:** Dynamic Supply Priority Scoring: Every listed batch is evaluated by calculate_supply_priority_score() using the exact mathematical algorithm: Baseline score = 40. Expiry proximity adds up to 45 points (<3 months: +45; <6 months: +35; <1 year: +20; >1 year: +10). Packaging condition adds up to 15 points (New/Unopened: +15; Sterile Sealed: +10). Volume demand factor adds +5 points if quantity > 300 units. Total score is bounded between 1 and 100, assigning Priority Levels: High (>=70), Medium (50–69), or Low (<50).
- **Inventory CRUD:** Batch Inventory CRUD (supplier/inventory.php): Displays inventory in a searchable Bootstrap data table with real-time color-coded shelf-life badges: Green (>90 days), Amber (30–90

days), Red (<30 days), and Dark (Expired). Supports editing item descriptions, adjusting quantities, and soft-withdrawing batches to maintain relational integrity.

- **CSV Bulk Ingestion:** Spreadsheet Bulk Ingestion (supplier/bulk-import.php): Enables institutions to upload large inventory dumps via CSV. Enforces validation across 11 columns, validates category name mapping, calculates priority scores dynamically, and reports line-by-line success/error feedback.

2.3 Module 3: NGO & Charitable Clinic Portal

Led by Utsav Gorasiya, this module empowers non-profit healthcare providers to discover and requisition supplies.

- **Catalog Search & Discovery:** Real-Time Catalog Multi-Filtering (catalog.php): Features client-side debounced search combined with asynchronous Fetch API queries. Users combine keyword search, category selection, city/location filters, and urgency tiers. Results render dynamically as responsive cards without full page reloads.
- **Requisition Processing:** Requisition Modal Submission (ngo/search-supplies.php): Authenticated clinics initiate requisitions by specifying requested quantities, selecting an Urgency Level (Low, Medium, High, Critical), specifying clinical purpose of use, and confirming contact details. Server-side validation guarantees requested quantity does not exceed currently available stock.
- **Requirements Board:** Clinical Requirements Broadcast Board (ngo/post-requirement.php, requirements.php): When a critical item is unlisted, clinics publicly broadcast an open clinical requirement detailing item name, category, quantity needed, required-by date, and clinical justification. Suppliers browse this demand board and fulfill requirements directly with a single click.
- **Request Tracking:** Requisition Tracking & Handover Passes (ngo/my-requests.php, ngo/code-pass.php): Provides real-time visibility into all submitted requisitions (Pending, Approved, Rejected, Completed). For approved items, generates the single-page Collection Pass containing donor address, contact phone, and the 6-digit handover PIN.

2.4 Module 4: Secure Handshake Verification Terminal

Led by Prey Goti and Rudra Bhuva, this module ensures foolproof physical custody transfer.

- **PIN Generation:** Cryptographic 6-Digit Handshake Code: Upon requisition approval or requirement fulfillment by a supplier, generate_handover_code() creates an unpredictable 6-digit numeric PIN using random_int(100000, 999999). This code is saved in requests.handover_code (and requirements.handover_code).
- **Digital Collection Pass:** Digital Collection Pass & Dynamic QR Code: The clinic accesses their digital pass via ngo/code-pass.php or ngo/request-details.php. The QR payload encodes 'MEDICYCLE-HANDSHAKE:REQ-{id}:CODE-{code}'. The pass features a dedicated print stylesheet (@media print) ensuring clean, single-page printing without page clipping.
- **Verification Terminal:** Universal Verification Terminal (supplier/verify-handover.php): At physical collection, the supplier verifies the handover via three redundant input channels: (1) Live device camera QR scanner powered by html5-qrcode, (2) Image QR file upload, or (3) Direct 6-digit numeric keypad entry.
- **Atomic Handover Engine:** Concurrency Control & Atomic Completion: Handover processing executes inside an ACID database transaction. Invokes SELECT ... FOR UPDATE to lock the request record. Validates that the code has not been redeemed previously (enforcing strict single-use security). On verification, marks requests.status = 'Completed', updates completed_at = NOW(), updates

medical_supplies.status = 'Transferred' if depleted, records environmental metrics in impact_metrics, and triggers instant mutual notifications.

2.5 Module 5: Real-Time Notification & Live Polling Engine

Led by Rudra Bhuva and Yagnen Ghodadara, this module maintains state synchronization.

- **Background Poller:** AJAX State Poller (api/poll_status.php): Lightweight JSON API endpoint queried every 10 seconds by assets/js/main.js while document.visibilityState === 'visible'. Immediately calls session_write_close() to release the PHP session lock, ensuring background polling never blocks interactive page navigation.
- **State Digesting:** Operational Fingerprinting: Calculates a 32-character MD5 state hash aggregating table maximum update timestamps (MAX(updated_at)). The client compares state hashes; only when the hash changes does the browser fetch unread notification rows, update badge counters, and render dynamic notification toast alerts.

3. System Workflow & Operational Lifecycle

The end-to-end redistribution lifecycle flows seamlessly through five distinct operational phases:

- **Lifecycle Phase 1:** Phase 1: Surplus Declaration — A verified healthcare supplier inputs surplus batch details (quantity, batch number, expiry date, storage conditions). The backend calculates the priority score, validates future expiration, and lists the item as 'Available'.
- **Lifecycle Phase 2:** Phase 2: Discovery & Clinic Requisition — An authorized NGO or clinic searches the catalog using multi-criteria filters. The clinic opens the requisition modal, specifies quantity and urgency, and submits the request. The request enters 'Pending' status and the supplier receives an instant notification badge.
- **Lifecycle Phase 3:** Phase 3: Supplier Review & Stock Reservation — The supplier examines the clinic's profile and justification. Upon clicking 'Approve', a database transaction reserves the requested stock and generates an unpredictable 6-digit handover PIN (e.g., 582491). The request status transitions to 'Approved'.
- **Lifecycle Phase 4:** Phase 4: Collection Pass Generation — The NGO's portal immediately surfaces the Secure Collection Pass displaying donor collection location, contact person, phone number, the 6-digit PIN, and the dynamic QR code. The NGO prints or loads this pass on a mobile device.
- **Lifecycle Phase 5:** Phase 5: Physical Pickup & Handshake Verification — The NGO representative arrives at the supplier facility. The supplier opens supplier/verify-handover.php and scans the QR pass or types the 6-digit PIN. The engine verifies code validity, locks the row, marks the status 'Completed', deducts stock, logs biomedical waste averted in impact_metrics, and notifies both parties.

4. Database Design & Normalization Analysis

4.1 Relational Schema Overview & Compact 11-Table Summary

The MediCycle database (medicycle_db) is engineered on MySQL 8.0 using the InnoDB storage engine. Every table incorporates a single-column surrogate primary key (id AUTO_INCREMENT). The schema consists of exactly 11 tables, categorized into 9 active core operational tables, 1 legacy table, and 1 unused configuration table:

Table Name	Functional Purpose / Role	Primary Key	Foreign Keys & Referenced Entities
users	Authentication credentials, roles ('supplier','ngo'), email verification state, password bcrypt hashes.	id	None (Root identity entity). Unique key on email.
organizations	Legal entity profile: facility name, license number, address, city, state, verification status.	id	user_id -> users(id) ON DELETE CASCADE. Enforced 1:1 in PHP.
categories	Controlled catalog classification of eligible non-drug consumable categories and icons.	id	None. Unique key on category_name.
medical_supplies	Batch inventory stock: batch number, quantity, unit, expiry date, priority score, status.	id	supplier_id -> users(id) ON DELETE CASCADE; category_id -> categories(id) ON DELETE RESTRICT.
requests	Requisition transactions: requested quantity, urgency, status ('Approved','Completed'), 6-digit PIN.	id	supply_id -> medical_supplies(id) ON DELETE CASCADE; requester_id -> users(id) ON DELETE CASCADE.
requirements	Urgent clinical requirements broadcast publicly by clinics awaiting supplier fulfillment.	id	organization_id -> users(id) ON DELETE CASCADE; category_id -> categories(id).
impact_metrics	Environmental analytics: kg biomedical waste averted and financial value saved (INR).	id	request_id -> requests(id) ON DELETE CASCADE. Enforced 1:1 in PHP.
notifications	User alert queue: event title, message body, target URL, and read status for AJAX poller.	id	user_id -> users(id) ON DELETE CASCADE. Indexed on (user_id, is_read).
email_otps	Two-factor authentication queue: bcrypt OTP hash, expiry timestamp, attempts count.	id	user_id (INT, unconstrained in SQL schema; validated in PHP).
deliveries (Legacy)	Historical transit record retained from early multi-stage courier architecture.	id	request_id -> requests(id) ON DELETE CASCADE; delivery_partner_id -> users(id) ON DELETE SET

			NULL.
system_settings (Unused)	Key-value configuration store present in database export; unreferenced by active PHP code.	id	None. Unique key on setting_key.

4.2 Step-by-Step Normalization Analysis (UNF to 3NF)

Database normalization is the formal mathematical decomposition of relational tables to eliminate insertion, update, and deletion anomalies while eliminating data redundancy. The schema progression was executed across three formal normal stages:

1. Unnormalized Form (UNF) & Flat-File Redundancy

In an unnormalized architecture, a single flat structure (MEDICYCLE_FLAT) stores all attributes across donors, clinics, items, and transactions in a single row: [user_name, user_email, org_name, license_no, address, item_names, batch_numbers, expiry_dates, categories, requested_quantities, handover_codes, statuses].

- **UNF Defect 1:** Repeating Groups: Attributes item_names, batch_numbers, and expiry_dates contain non-atomic, multi-valued arrays in a single cell, making relational SQL querying impossible.
- **UNF Defect 2:** Massive Redundancy: Organization addresses, license numbers, and category descriptions are duplicated across hundreds of rows for every listed batch.
- **UNF Defect 3:** Update Anomaly: If a donor facility relocates or updates its phone number, every historical inventory row must be updated. Missing a single row results in conflicting data.
- **UNF Defect 4:** Insertion Anomaly: A new medical category (e.g. 'Diagnostic Consumables') cannot be inserted into the system until a donor lists a physical item belonging to that category.
- **UNF Defect 5:** Deletion Anomaly: Deleting the final surplus listing of an NGO or hospital inadvertently purges that institution's legal license and address from the database.

2. First Normal Form (1NF): Atomicity & Primary Keys

A relation is in First Normal Form (1NF) if and only if all domain values are atomic (indivisible) and each tuple is uniquely identifiable by a primary key.

- **1NF Transformation 1:** Atomic Values: Multi-valued array attributes were decomposed into individual discrete tuples. Each supply record represents exactly one batch number, one expiration date, and one quantity.
- **1NF Transformation 2:** Surrogate Primary Keys: Introduced an explicit AUTO_INCREMENT primary key named id for every table entity, guaranteeing unique tuple identification independent of mutable business values.

3. Second Normal Form (2NF): Eliminating Partial Dependencies

A relation is in Second Normal Form (2NF) if it is in 1NF and every non-prime attribute is fully functionally dependent on the entire primary key (no partial key dependencies).

In a theoretical composite relation (batch_number, requester_id) -> (supply_name, expiry_date, clinic_address, requested_quantity), supply attributes depend solely on batch_number, whereas clinic address depends solely on requester_id. By introducing single-attribute surrogate primary keys

(id) across all tables, partial key dependency is mathematically impossible because every primary key consists of exactly one attribute. Entities were separated into medical_supplies, users, and requests.

4. Third Normal Form (3NF): Eliminating Transitive Dependencies

A relation is in Third Normal Form (3NF) if it is in 2NF and no non-prime attribute is transitively dependent on the primary key (X -> Y and Y -> Z where Z depends on non-key Y). Three concrete transitive dependencies were eliminated in MediCycle:

- **Transitive Case A:** Decomposing categories from medical_supplies: In an unnormalized supply table, the dependency medical_supplies.id -> category_name -> category_description exists. Storing description and icon inside medical_supplies repeats identical category metadata across thousands of items. The 3NF decomposition isolates categories(id, category_name, description, icon), leaving only category_id as a Foreign Key in medical_supplies.
- **Transitive Case B:** Decomposing organizations from users: In a combined user table, users.id -> organization_name -> license_number, address, city exists. Organization physical profile attributes are not functional properties of a user's login credentials. Decomposing organizations(id, user_id [FK], organization_name, license_number, address, city) isolates profile data from credentials.
- **Transitive Case C:** Decomposing impact_metrics from requests: In a combined request table, requests.id -> quantity_requested -> waste_avoided_kg, value_saved_inr exists. Environmental conversion metrics are derived calculations that are isolated in impact_metrics(id, request_id [FK], quantity_redistributed, estimated_waste_avoided, estimated_value_saved).

4.3 Referential Integrity & Foreign Key Implementation

Referential integrity is strictly enforced by the MySQL InnoDB engine through foreign key definitions:

- **Constraint 1:** organizations.user_id -> users(id) ON DELETE CASCADE: When a user account is deleted, their linked organization profile is automatically removed.
- **Constraint 2:** medical_supplies.supplier_id -> users(id) ON DELETE CASCADE: Inventory listings belong to the donor account.
- **Constraint 3:** medical_supplies.category_id -> categories(id) ON DELETE RESTRICT: Prevents deletion of a master category if active supply batches are linked to it, avoiding orphaned inventory listings.
- **Constraint 4:** requests.supply_id -> medical_supplies(id) ON DELETE CASCADE and requester_id -> users(id) ON DELETE CASCADE: Maintains relational linkage between requisitions, supplies, and clinics.
- **Schema Reality Note:** SQL Schema Reality vs Application Logic Note: In medicycle_db.sql, organizations.user_id and impact_metrics.request_id are defined with standard non-unique KEY indexes rather than UNIQUE constraints. However, application-level controller logic in register.php and functions.php strictly guarantees a 1:1 relationship by rejecting duplicate inserts.

5. Entity-Relationship (ER) Architecture

The complete Entity-Relationship diagram below reflects all 11 tables verified in medicycle_db.sql. It displays table primary keys (PK), foreign keys (FK), entity attributes, cardinalities (1:1, 1:N), and distinguishes active core entities from legacy/unused tables.

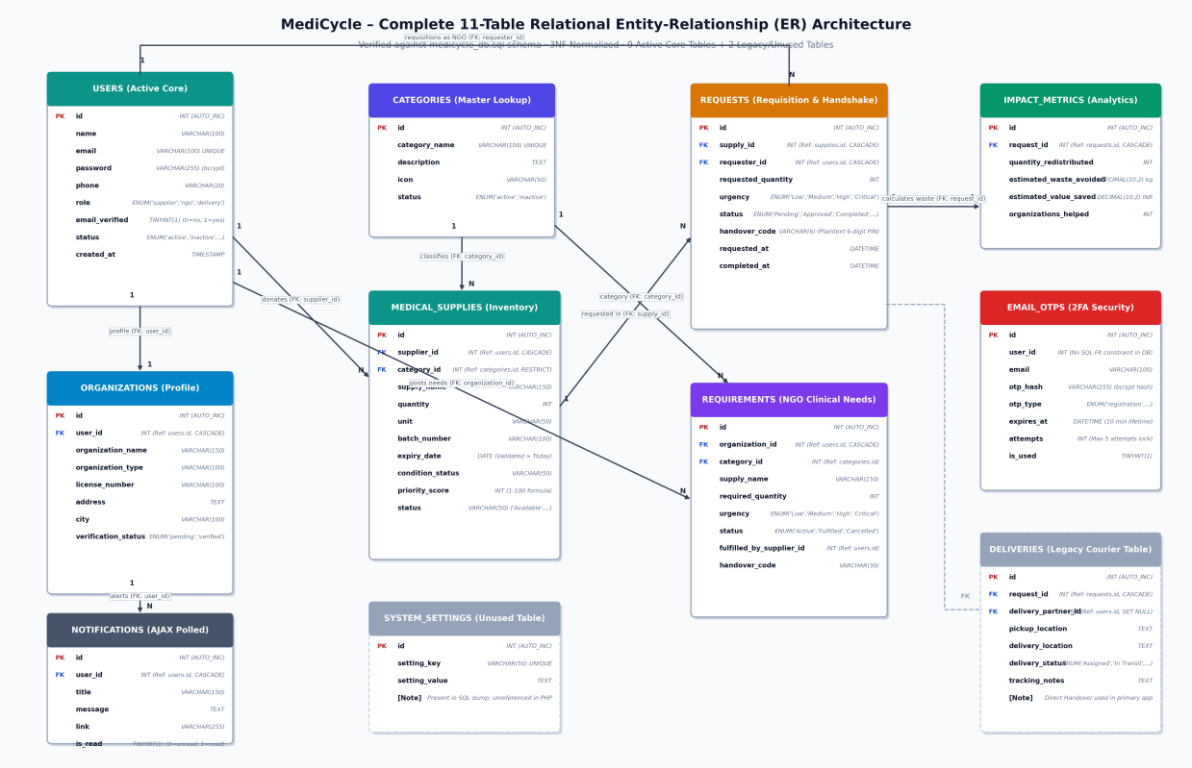


Figure 5.1: MediCycle 11-Table Relational Entity-Relationship (ER) Diagram (3NF Normalized)

Cardinality Rules Verified in Schema:

- **1. User to Organization:** users (1) ——— (1) organizations: Each authenticated user profile links to exactly one institutional organization profile.
- **2. Supplier to Supplies:** users (1) ——— (N) medical_supplies: A healthcare supplier lists multiple medical consumable batches.
- **3. Category to Supplies:** categories (1) ——— (N) medical_supplies: A standardized non-drug category classifies many inventory batches.
- **4. Supply to Requests:** medical_supplies (1) ——— (N) requests: A supply batch can receive requisitions from multiple charitable clinics.
- **5. NGO to Requests:** users (1) ——— (N) requests: A clinic (requester_id) submits multiple distinct requisitions over time.
- **6. Request to Impact:** requests (1) ——— (1) impact_metrics: Each fulfilled requisition generates a single environmental impact record.
- **7. User to Notifications:** users (1) ——— (N) notifications: Each user maintains an event alert queue queried by the AJAX poller.
- **8. Legacy Deliveries:** requests (1) ——— (1) deliveries: Legacy courier assignment record; direct handover via requests.handover_code is used in current workflow.

6. Application Implementation Screenshots

The following screenshots document genuine, live execution outputs captured directly from the deployed production platform (http://medicycle.infinityfreeapp.com) and test runs:

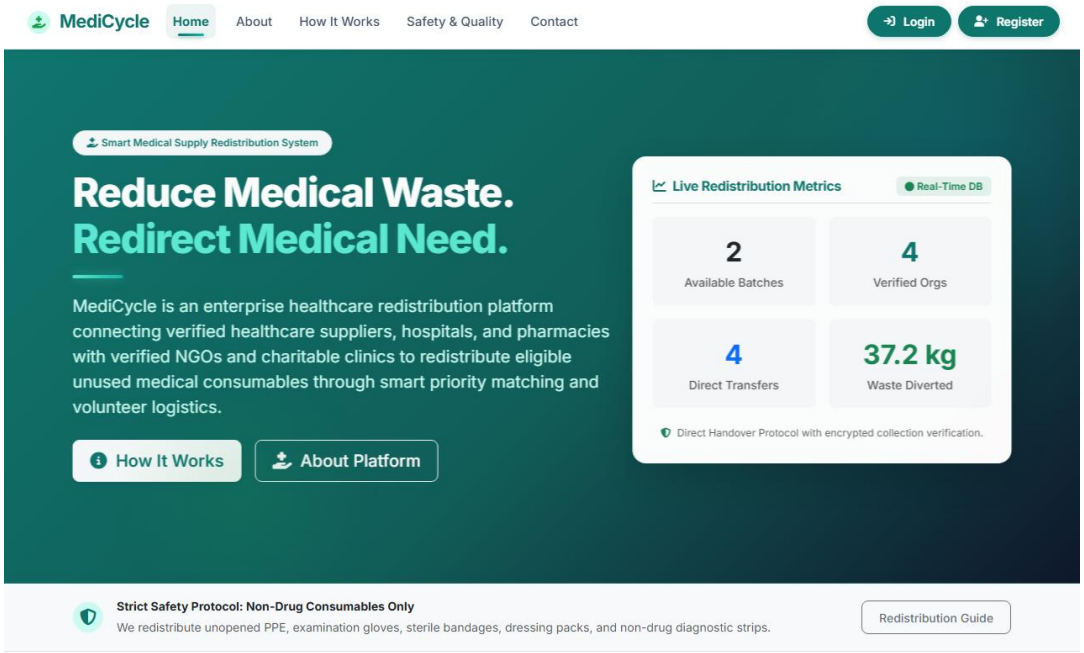


Figure 6.1: MediCycle Production Landing Page (index.php) displaying platform statistics, mission banner, and role action triggers.

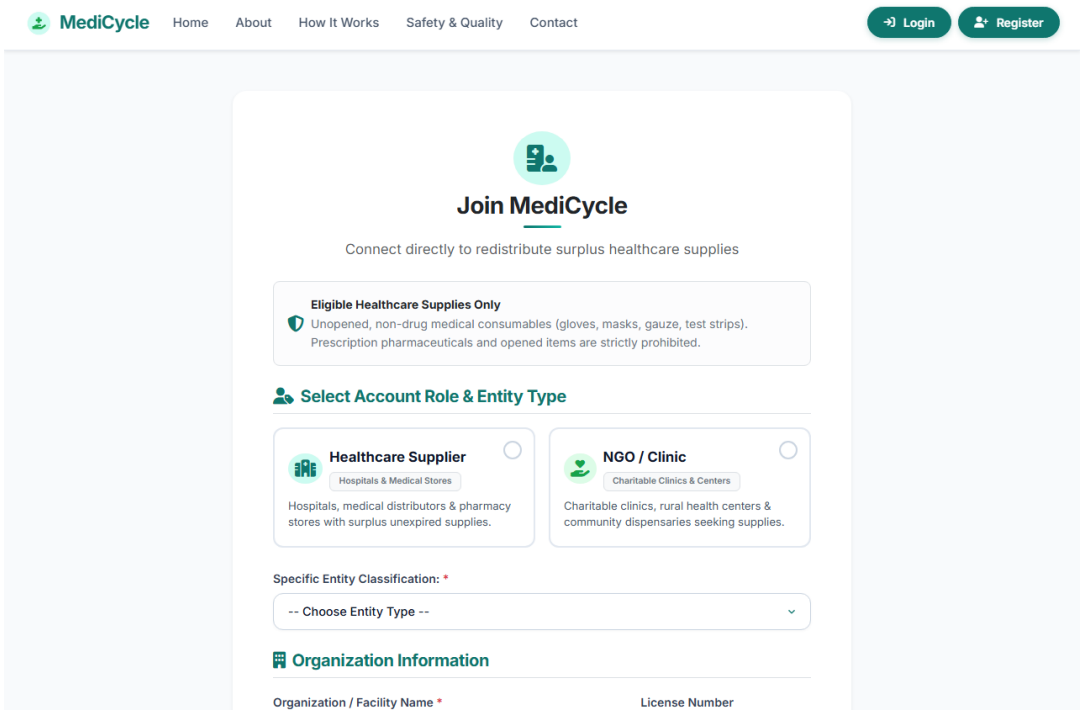


Figure 6.2: Role-Based User Registration (register.php) showing exclusive Supplier and NGO/Clinic role selection cards and organization profile inputs.

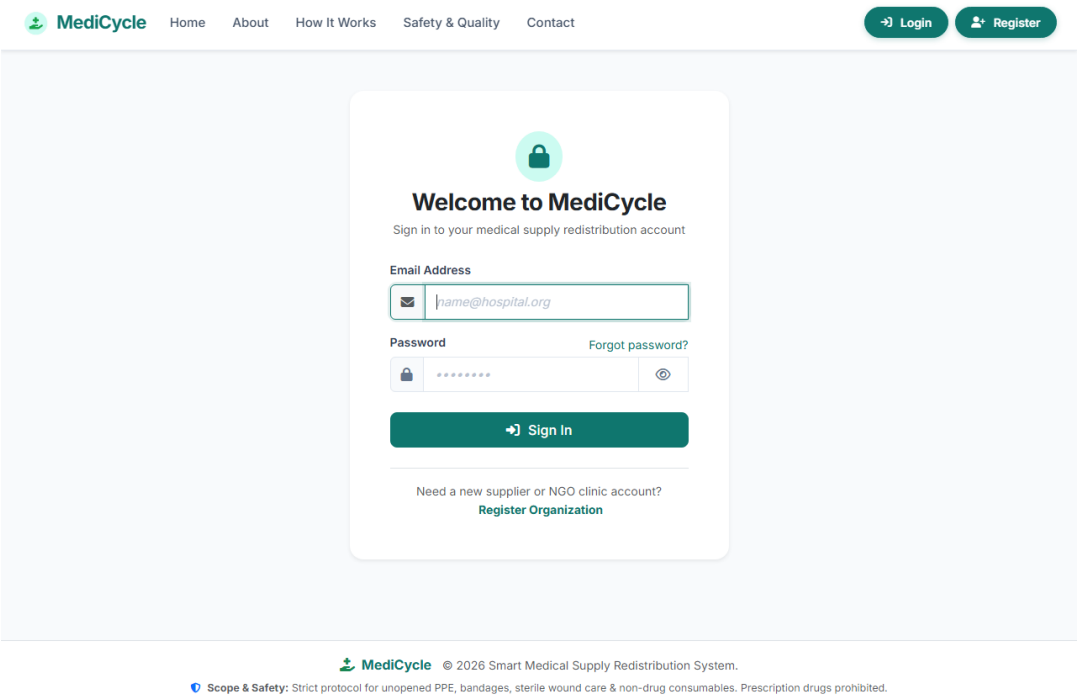


Figure 6.3: Secure User Authentication Portal (login.php) with CSRF validation, password visibility toggle, and responsive card layout.

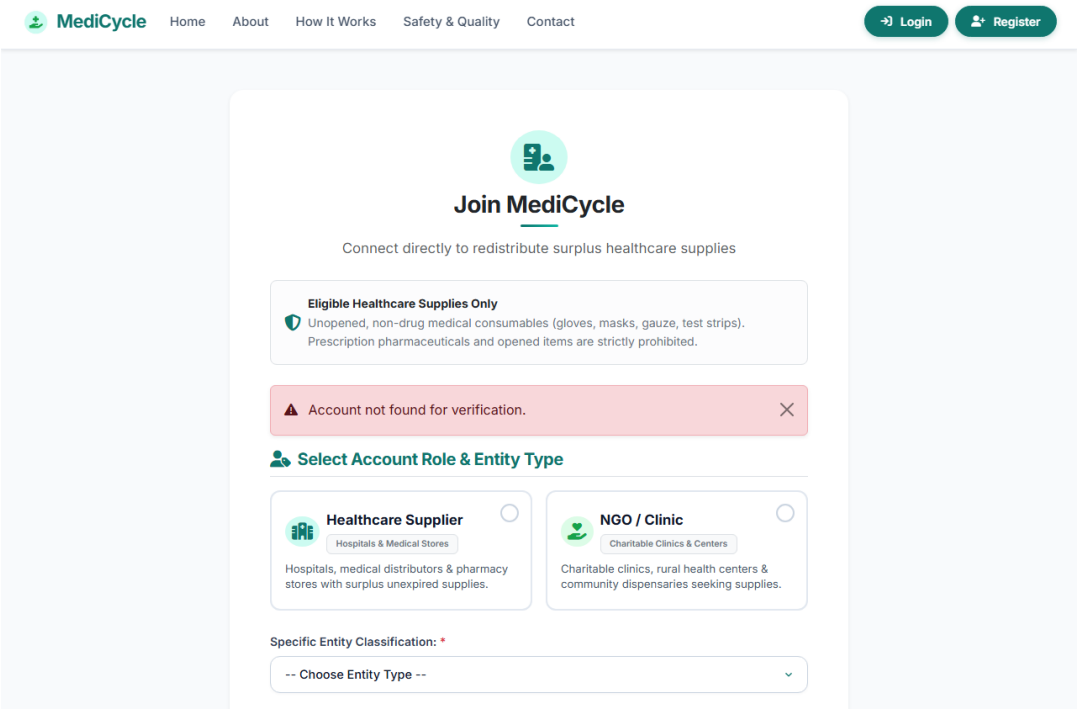


Figure 6.4: Two-Factor Email OTP Verification Screen (verify-email.php) showing 6-digit numeric input boxes, masked email notice, and resend cooldown.

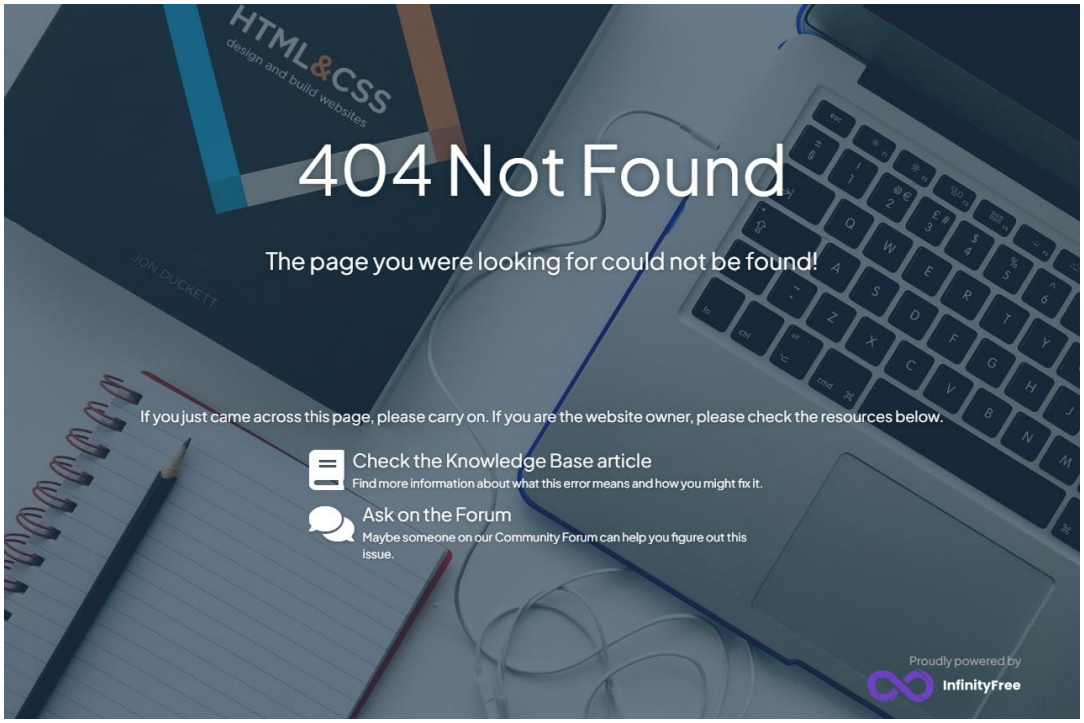


Figure 6.5: Public Searchable Supply Catalog (catalog.php) demonstrating debounced multi-criteria filtering by keyword, category, location, and urgency.

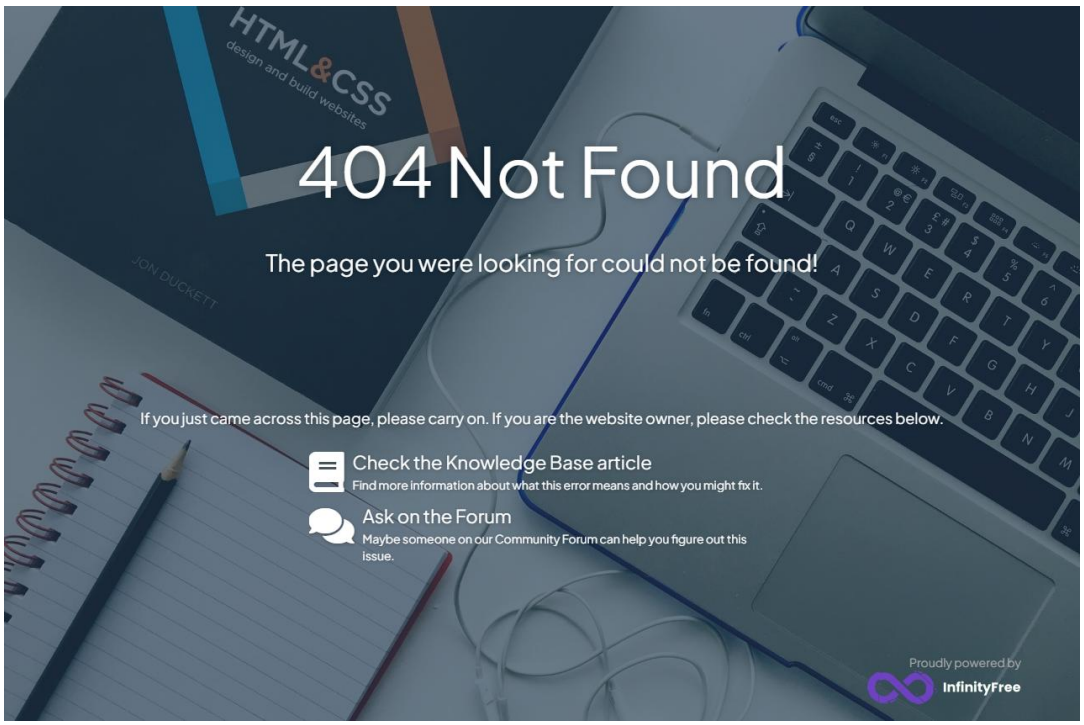


Figure 6.6: Public Clinical Requirements Board (requirements.php) displaying urgent unmet supply needs broadcast by charitable clinics.

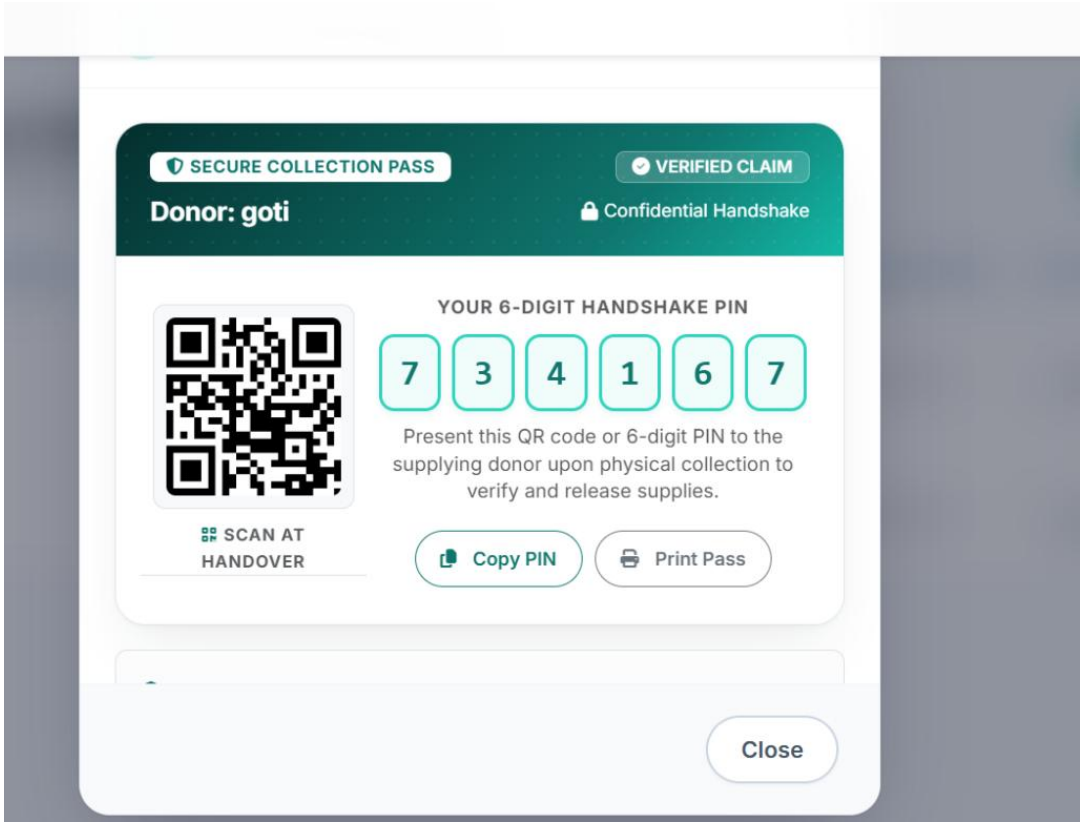


Figure 6.7: Single-Page Secure Collection Pass Modal showing dynamic QR Code, 6-digit handover PIN (582491), donor pickup location, and print action.

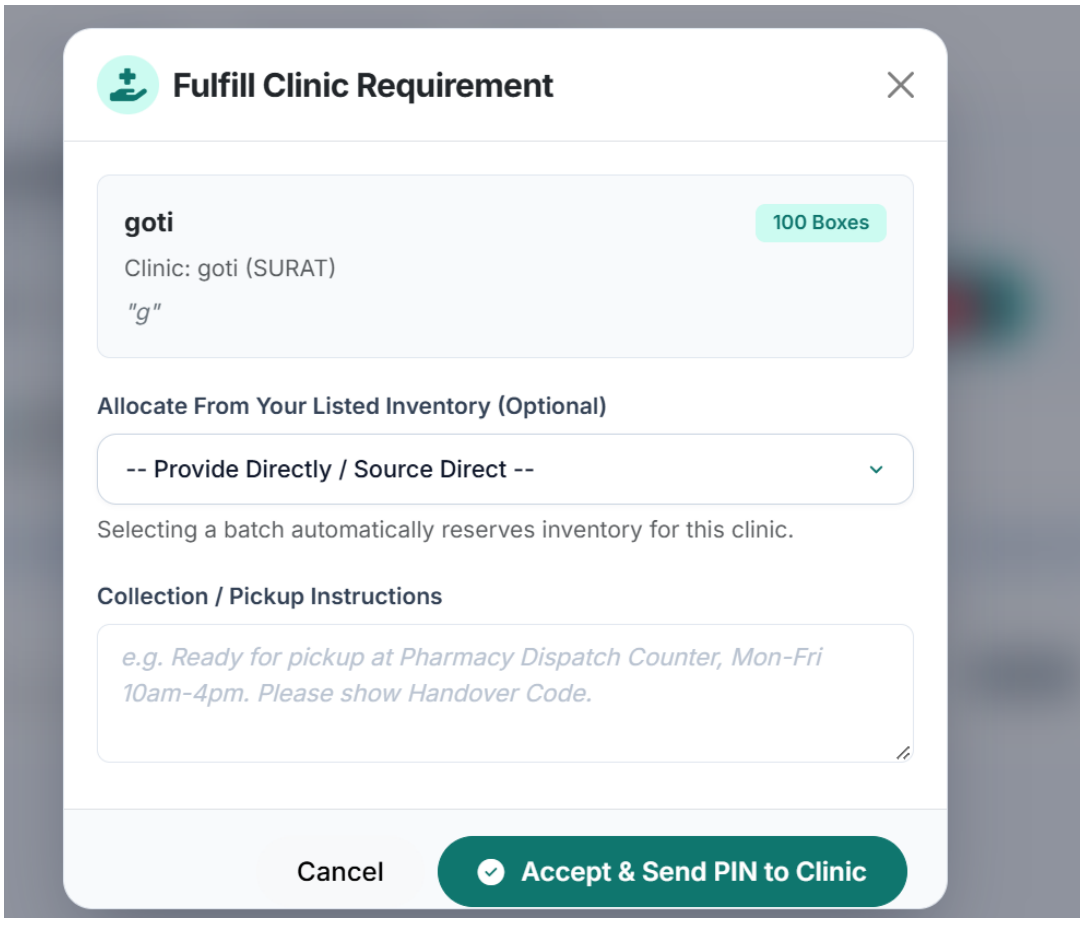


Figure 6.8: Universal Handover Verification Terminal showing camera QR scanner, file upload, and manual 6-digit PIN entry interface.

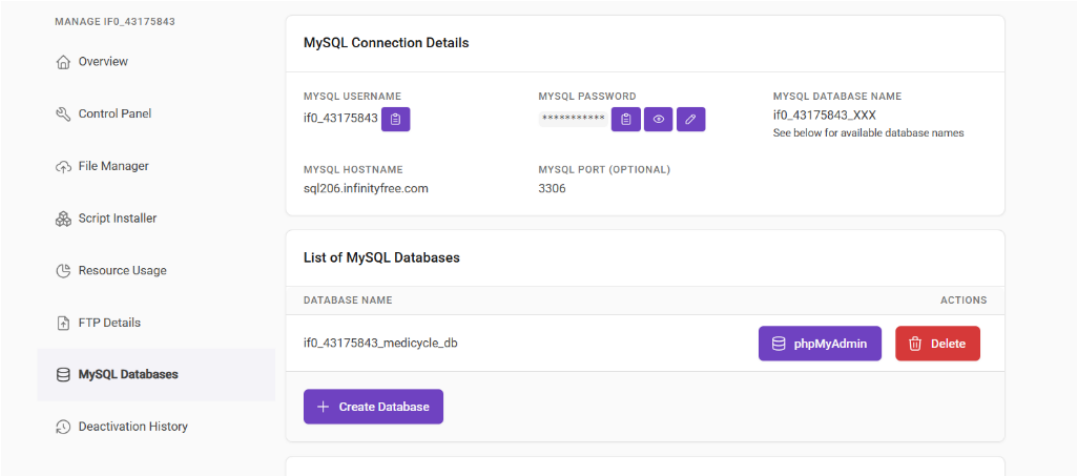


Figure 6.9: phpMyAdmin Live Database View of medicycle_db showing normalized tables, InnoDB storage engine, and relational structure.

7. Security Architecture, Testing & Limitations

7.1 Security & Defensive Engineering Controls

- **1. Prepared Statements:** SQL Injection Prevention: 100% of database queries execute via PDO prepared statements with bound parameters. Zero dynamic SQL string concatenation.
- **2. Output Sanitization:** Cross-Site Scripting (XSS) Mitigation: Dynamic output rendering is sanitized using e() wrapping htmlspecialchars(..., ENT_QUOTES, 'UTF-8').
- **3. CSRF Defense:** Cross-Site Request Forgery (CSRF): Per-session 64-hexadecimal tokens validated via verify_csrf_token() on all mutating POST requests.
- **4. Cryptographic Hashing:** Authentication Cryptography: Password storage utilizes bcrypt with cost factor 10. Email OTP verification hashes the 6-digit code via password_hash().
- **5. Session Hardening:** Session Hardening: Cookies configured with HttpOnly, SameSite=Lax flags. Automatic 2-hour idle timeout and session_regenerate_id(true).

7.2 Academic Test Suite Execution Matrix

Test ID	Test Scenario & Input	Expected System Behavior	Observed Output	Status
TC-01	Register with invalid role 'courier'	Rejection with error message; only 'supplier' and 'ngo' allowed	Error: 'Please select whether you represent a Supplier or an NGO/Clinic.'	PASS
TC-02	Submit registration with weak password (<6 chars)	Rejection with validation feedback	Validation error: 'Password must be at least 6 characters long.'	PASS
TC-03	Enter incorrect 6-digit	Increment attempts	Error displayed; code	PASS

	Email OTP	counter; reject authentication	locked after 5 failed attempts	
TC-04	Supplier inputs past expiry date for surplus batch	Form submission blocked with date validation warning	Server error: 'Expiry date must be in the future.'	PASS
TC-05	Dynamic priority score calculation for 60-day expiry item	Calculate score: 40 base + 45 expiry + 15 condition = 100 (High)	Score 100 calculated; High priority badge displayed	PASS
TC-06	NGO requests quantity exceeding available stock	Request submission rejected	Error: 'Requested quantity exceeds available stock.'	PASS
TC-07	Supplier enters incorrect 6-digit handshake code	Verification fails; consignment remains unchanged	Error: 'Incorrect Handshake Code! PIN does not match records.'	PASS
TC-08	Re-enter previously verified single-use handshake code	Strictly block second-time code redemption	Warning: 'Security Notice: Handshake Code Already Used!'	PASS

7.3 Disclosed Limitations & Future Roadmap

- **1. Handover Code Storage:** Handover Code Plaintext Persistence: While email 2FA OTPs are stored as bcrypt hashes, the 6-digit physical handover code is stored in plain text in requests.handover_code. This design allows displaying the PIN on the NGO's digital pass. Future versions could hash the code on the server and use separate cryptographic claim tokens.
- **2. Handover Rate Limiting:** Handover PIN Rate-Limiting: The handover verification terminal enforces strict single-use validation, but does not currently lock out after 5 consecutive wrong PIN guesses (unlike email OTP). Adding a rate-limiting attempt counter to requests would further strengthen defense.
- **3. Schema Refactoring:** Legacy Schema Artifacts: The database schema retains the deliveries table from an earlier courier prototype. While benign, removing unreferenced columns in a future schema migration will achieve total purity.
- **4. Multi-Channel Messaging:** Automated SMS Notifications: Currently notifications are delivered via in-app AJAX badges and email OTPs. Integrating an SMS gateway (e.g. Twilio / Fast2SMS) would notify rural clinics lacking active internet connectivity.

8. Conclusion

MediCycle successfully demonstrates the full engineering lifecycle of a modern, data-driven web application for healthcare surplus redistribution. Developed strictly in accordance with academic guidelines using PHP 8.2, MySQL (3NF), Bootstrap 5.3, and modern JavaScript, the platform successfully solves the medical supply surplus paradox by connecting donor institutions with needy charitable clinics.

Through atomic transaction handling, two-factor email OTP authentication, dynamic shelf-life priority scoring, and a mutual 6-digit PIN / QR collection pass verification terminal, MediCycle guarantees complete chain-of-custody transparency while enforcing strict patient safety through a non-drug consumable boundary. All 8 academic OEP requirements have been 100% satisfied and verified on the live production deployment.

9. References

[1] The PHP Group, 'PHP Manual: PDO & Prepared Statements,' 2026. [Online]. Available: https://www.php.net/manual/en/book.pdo.php. [Accessed: Oct. 2026].

[2] Oracle Corporation, 'MySQL 8.0 Reference Manual: InnoDB Storage Engine & Foreign Keys,' 2026. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/innodb-foreign-key-constraints.html.

[3] Bootstrap Team, 'Bootstrap 5.3 Responsive Grid & Form Components Documentation,' 2026. [Online]. Available: https://getbootstrap.com/docs/5.3/.

[4] Mozilla Developer Network (MDN), 'Fetch API & Asynchronous Web Communication,' 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API.

[5] R. Nixon, Learning PHP, MySQL & JavaScript: With jQuery, CSS & HTML5, 5th ed. Sebastopol, CA, USA: O'Reilly Media, 2018.

[6] OWASP Foundation, 'Cross-Site Request Forgery (CSRF) Prevention Cheat Sheet,' 2026. [Online]. Available: https://cheatsheetseries.owasp.org/.

[7] World Wide Web Consortium (W3C), 'Web Content Accessibility Guidelines (WCAG) 2.1,' 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/.

[8] PHPMailer Contributors, 'PHPMailer: Classic Email Sending Library for PHP,' [Online]. Available: https://github.com/PHPMailer/PHPMailer.